

Loom Video Walkthrough Script & Guide

Technical Walkthrough for Artikate Studio — AI/ML/LLM Engineer Assessment

1. Preparation (Before Recording)

- **Open your tools:** Open your terminal (PowerShell) and a browser window side-by-side.
- **Clear previous states:** Close old browser tabs so you have a fresh environment.
- **Verify the .env file:** Ensure your **GROQ_API_KEY** is active in the **.env** file.

2. Step 1: Introduction (0:00 - 1:00)

On-Screen: Codebase open in VS Code / IDE.

Narration Script:

"Hello team, my name is Sanjeev. In this video, I will show a quick walkthrough of my technical implementation for the Artikate Studio AI/ML/LLM Engineer assessment. I will demonstrate the live RAG application, source citations, hallucination refusal behavior, the automated evaluation script, and the high-throughput ticket classifier."

3. Step 2: Run RAG App & Live Query (1:00 - 2:30)

Action: Launch the Streamlit application in a fresh terminal:

```
$env:PYTHONPATH="."
.venv\Scripts\python -m streamlit run src/section2/app.py
```

Action: Open your web browser at **http://localhost:8501**.

Narration Script:

"First, I am launching the Streamlit web application. As you can see, I have designed a premium glassmorphic dark UI called **LexAI**. On the sidebar, we can see the status badge indicating that our **Groq LLM Llama 3.3 70B** is active using the API key from our **.env** file."

Let's run a live query to find the governing law in the pre-loaded Vendor NDA. I will ask: **'What is the governing law in the Vendor NDA?'**.

Here is the generated answer: it correctly identifies it as the State of Delaware. Beneath the answer, the UI displays clickable **Source Citation Cards** showing the exact file name and page number. If we expand

the **Retrieved Context Chunks**, we can inspect the exact text blocks retrieved by our FAISS index, along with our calculated semantic confidence score."

4. Step 3: Refusal & Hallucination Mitigation (2:30 - 3:45)

Action: Type an out-of-context question like: 'Who is the current Prime Minister of India?'

Narration Script:

"Now, let's test how the pipeline handles questions where context is missing or irrelevant. I will ask it about the current Prime Minister of India, which is not in our legal documents corpus.

As you can see, the app immediately refuses to answer, saying: '**I cannot find the answer in the provided context.**'

Under the hood, this refusal is driven by our **3-tier safety architecture**: first, we compute a cosine similarity score. If the best chunk falls below our threshold, the system flags it. Second, the LLM system prompt strictly binds it to the retrieved context. Third, we calculate a final confidence rating. This prevents the LLM from hallucinating answers to general-knowledge questions."

5. Step 4: Run RAG Evaluation Harness (3:45 - 5:00)

Action: Switch to terminal (Ctrl+C to stop Streamlit, or new tab) and run:

```
$env:PYTHONPATH="."
.venv\Scripts\python src/section2/evaluate.py
```

Narration Script:

"Next, I am running the automated evaluation harness for Section 2. This runs a dataset of 10 ground-truth legal QA pairs against our pipeline. It evaluates the retrieval component and computes the **Precision@3** metric.

The evaluation has finished. The pipeline achieved a **Precision@3 score of 90.00% (9/10)**. The only case it refused to answer was the out-of-context test question, which is exactly the desired behavior to prevent hallucinations."

6. Step 5: Ticket Classification Benchmarks (5:00 - 6:30)

Action: Run the Section 3 classifier evaluation script in the terminal:

```
$env:PYTHONPATH="."
.venv\Scripts\python src/section3/evaluate_classifier.py
```

Narration Script:

*"For Section 3, we built a Ticket Classifier to handle 2,880 tickets per day. I designed a hybrid approach: we encode tickets using local **all-MiniLM-L6-v2** embeddings, and feed them into a Logistic Regression classifier optimized using 5-Fold Cross-Validation.*

*Running our evaluation script, we see an **Overall Accuracy of 92.00%** on our 100-item test set. The confusion analysis shows that the most confused classes are **complaint** and **billing** (confused 2 times), due to semantic overlap in payment complaints.*

*In terms of speed, our inference latency is around **30ms per ticket**, which is 16 times faster than the required 500ms constraint, allowing us to easily handle peak throughput on a single CPU core."*

7. Step 6: Conclusion (6:30 - 7:00)

Narration Script:

"All code is clean, fully documented, and pushed to my GitHub repository. In ANSWERS.md and DESIGN.md, I have documented all diagnosis logs, system designs, and security details. Thank you very much for your time and review."